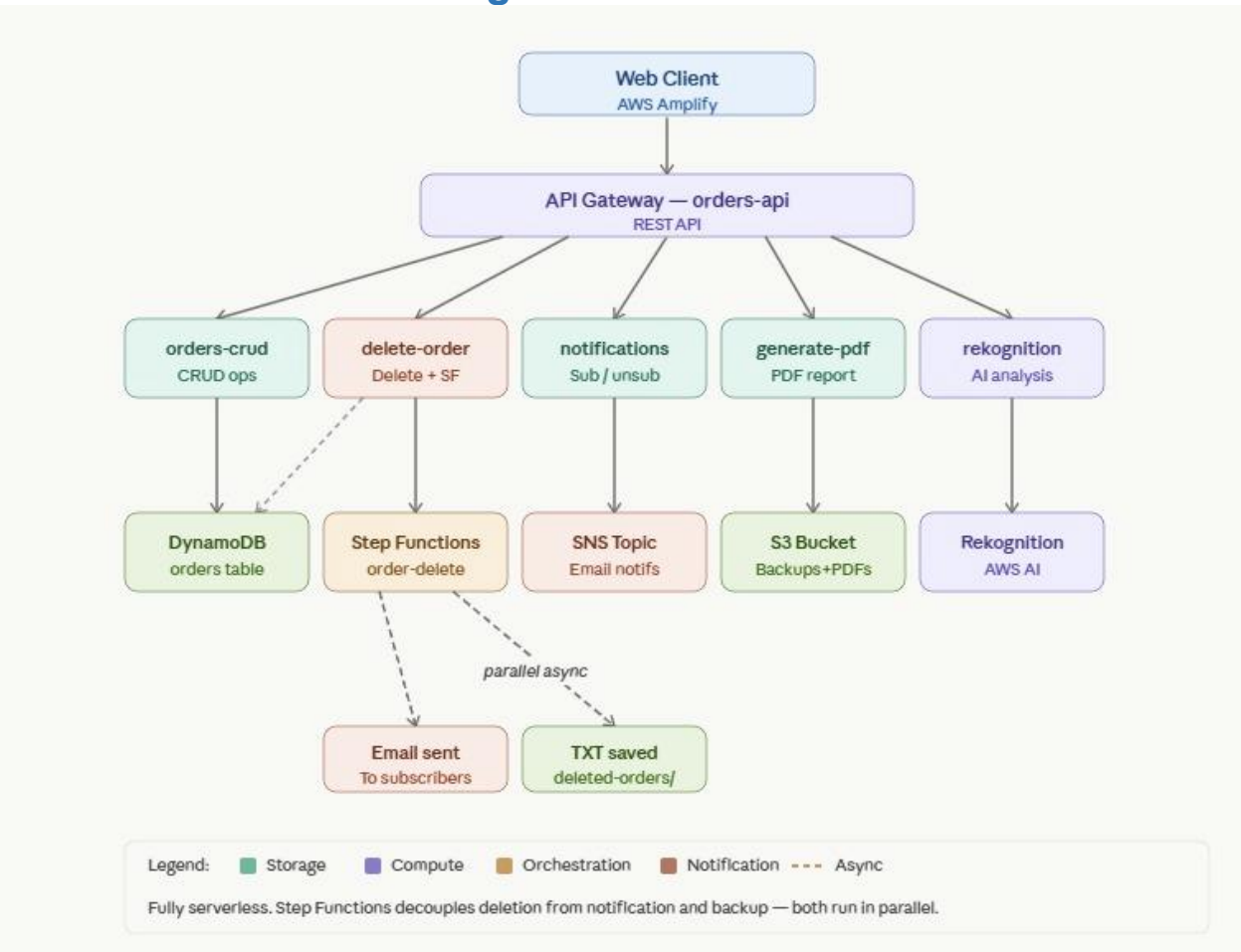
Event-Driven Serverless Order Management System

Submitted by: Adi Avramov and Adi Shacham

1. AWS Architecture Diagram



Architecture Overview

The system is built on a fully serverless, event-driven architecture using the following AWS services:

- API Gateway - Exposes all REST endpoints to the client
- AWS Lambda - Business logic for all operations (orders-crud, delete-order, notifications, generate-pdf, rekognition)
- DynamoDB - Persistent storage for all orders
- Step Functions - Orchestrates the asynchronous delete process (SNS + S3 in parallel)
- SNS - Sends email notifications to subscribed users when an order is deleted
- S3 - Stores deleted order backups as TXT files and generated PDF reports
- AWS Rekognition - AI-powered image analysis for the freestyle enhancement
- AWS Amplify - Hosts the web client (HTML/CSS/JS)

Flow Description

1. Client sends HTTP requests to API Gateway.
2. API Gateway routes to the appropriate Lambda function.
3. Lambda functions interact with DynamoDB for data storage.
4. When an order is deleted, the delete-order Lambda immediately returns a response and asynchronously triggers a Step Function.
5. The Step Function runs two parallel branches: one publishes to SNS (email notification), the other saves a TXT backup to S3.
6. The generate-pdf Lambda reads all TXT files from S3 and creates a PDF report on demand.

Web Application URL

The web application is hosted on AWS Amplify and is publicly accessible at:

https://staging.dlgeznd7l7vj3.amplifyapp.com

2. AWS Setup Per Service

AWS Service	Why this service? / Functionality	CLI Command
DynamoDB	NoSQL database for persistent order storage. Chosen for its serverless, scalable nature. Table: orders with order_id (Partition Key) and creation_date (Sort Key) to support efficient querying and sorting by date.	aws dynamodb describe-table --table-name orders
Lambda (orders-crud)	Handles all CRUD operations for orders: Create, Get All, Get One, Update. Single function handles all HTTP methods (POST, GET, PUT).	aws lambda get-function --function-name orders-crud
Lambda (delete-order)	Dedicated Lambda for order deletion. Deletes from DynamoDB and asynchronously triggers Step Function. Kept separate as required by the project deliverables.	aws lambda get-function --function-name delete-order
Lambda (notifications)	Handles Subscribe and Unsubscribe operations via SNS. Registers/removes email addresses from the SNS topic.	aws lambda get-function --function-name notifications
Lambda (generate-pdf)	Reads all TXT files from S3, generates a PDF summary report using fpdf2, uploads it to S3, and returns a presigned URL valid for 7 days.	aws lambda get-function --function-name generate-pdf
Lambda (rekognition)	Freestyle enhancement. Receives a base64 image, calls AWS Rekognition to detect labels (objects, scenes), and returns them to the client.	aws lambda get-function --function-name rekognition
API Gateway	REST API that exposes all endpoints to the web client. Routes HTTP requests to the appropriate Lambda functions.	aws apigateway get-rest-apis
Step Functions	Orchestrates the asynchronous delete process. Runs SNS notification and S3 backup in parallel, ensuring neither blocks the delete response.	aws stepfunctions describe-state-machine --state-machine-arn arn:aws:states:us-east-1:339713037456:stateMachine:order-delete-process
SNS	Sends email notifications to subscribed users when an order is deleted. Users subscribe/unsubscribe via REST API.	aws sns list-topics
S3	Object storage for: TXT backups of deleted orders in deleted-orders/ folder, Generated PDF reports in reports/ folder.	aws s3 ls s3://orders-backup-bucket-26/
AWS Rekognition	AI image analysis service. Detects objects, scenes, and labels in uploaded images. Used as the freestyle enhancement.	aws rekognition list-collections

AWS Amplify	Hosts the web client (HTML/CSS/JS). Provides a public URL for the application.	aws amplify list-apps

3. APIs List

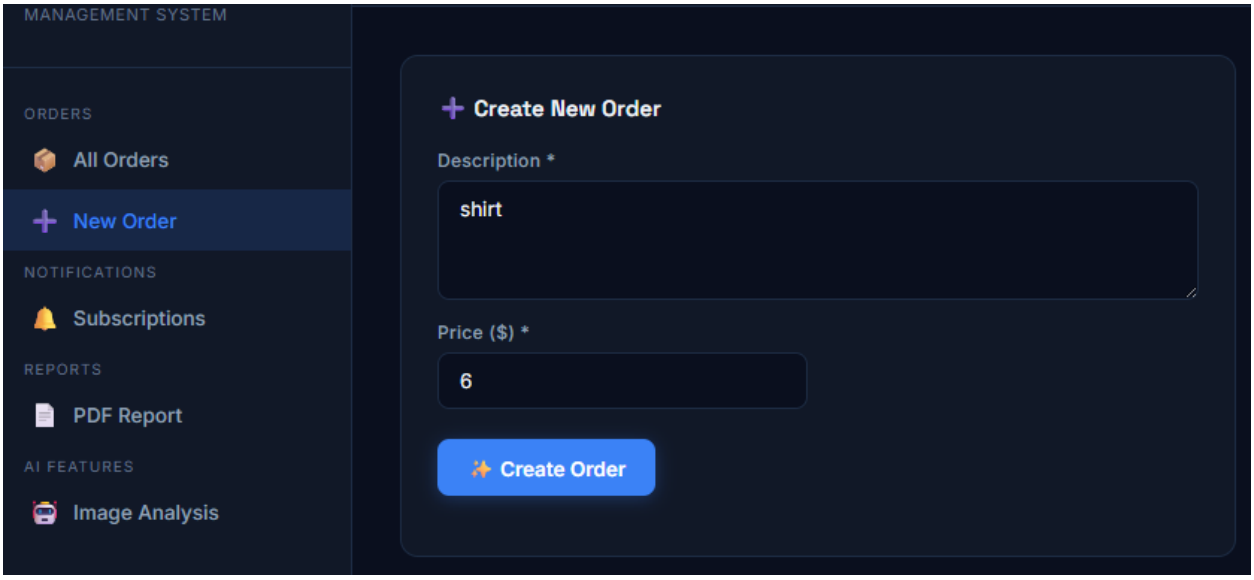
API Name	Method	URL	Sample Input	Sample Output
Create Order	POST	https://11u3e46zy4.execute-api.us-east-1.amazonaws.com/prod/orders	{"description": "Office Chair", "price": 250}	{"order_id": "abc-123", "creation_date": "2026-06-12T...", "price": 250, "description": "Office Chair"}
Get All Orders	GET	https://11u3e46zy4.execute-api.us-east-1.amazonaws.com/prod/orders	No body required	[{"order_id": "abc-123", "description": "Office Chair", "price": 250, ...}]
Get Specific Order	GET	https://11u3e46zy4.execute-api.us-east-1.amazonaws.com/prod/orders/{order_id}	URL: /orders/abc-123	{"order_id": "abc-123", "description": "Office Chair", "price": 250, ...}
Update Order	PUT	https://11u3e46zy4.execute-api.us-east-1.amazonaws.com/prod/orders/{order_id}	{"price": 300} or {"description": "New Desc"} — one field optional	{"message": "Order updated successfully"}
Delete Order	DELETE	https://11u3e46zy4.execute-api.us-east-1.amazonaws.com/prod/orders/{order_id}	URL: /orders/abc-123	{"message": "Order deleted successfully and archiving process started."}
Subscribe	POST	https://11u3e46zy4.execute-api.us-east-1.amazonaws.com/prod/notifications/subscribe	{"email": "user@example.com"}	"Subscription confirmation sent to user@example.com"
Unsubscribe	POST	https://11u3e46zy4.execute-api.us-east-1.amazonaws.com/prod/notifications/unsubscribe	{"email": "user@example.com"}	"Unsubscribed successfully"
Generate PDF	GET	https://11u3e46zy4.execute-api.us-east-1.amazonaws.com/prod/reports	No body required	{"message": "PDF generated successfully", "pdfUrl": "https://..."}
Analyze Image (Rekognition)	POST	https://11u3e46zy4.execute-api.us-east-1.amazonaws.com/prod/rekognition	{"image": "<base64_encoded_image>"}	{"labels": [{"Name": "Clothing", "Confidence": 99.9}, ...]}

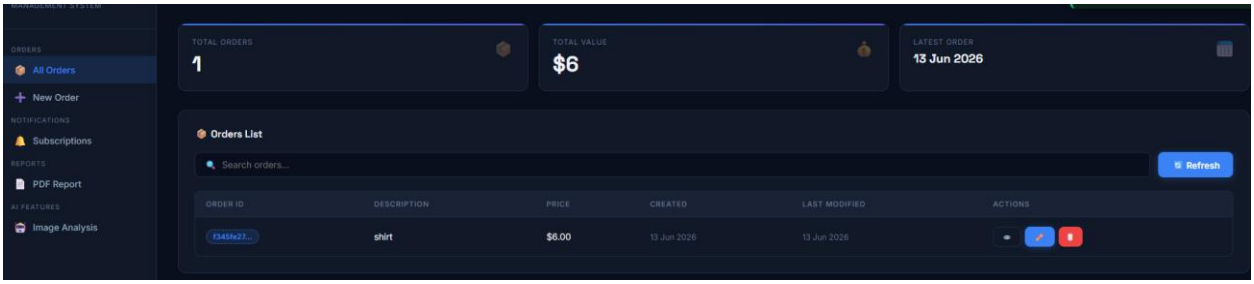
4. List of Tested Flows

Flow 1: Create Order

Steps: Navigate to 'New Order' tab → Enter description and price → Click 'Create Order'

Expected Result: Order is created and appears in the orders list sorted by creation date.

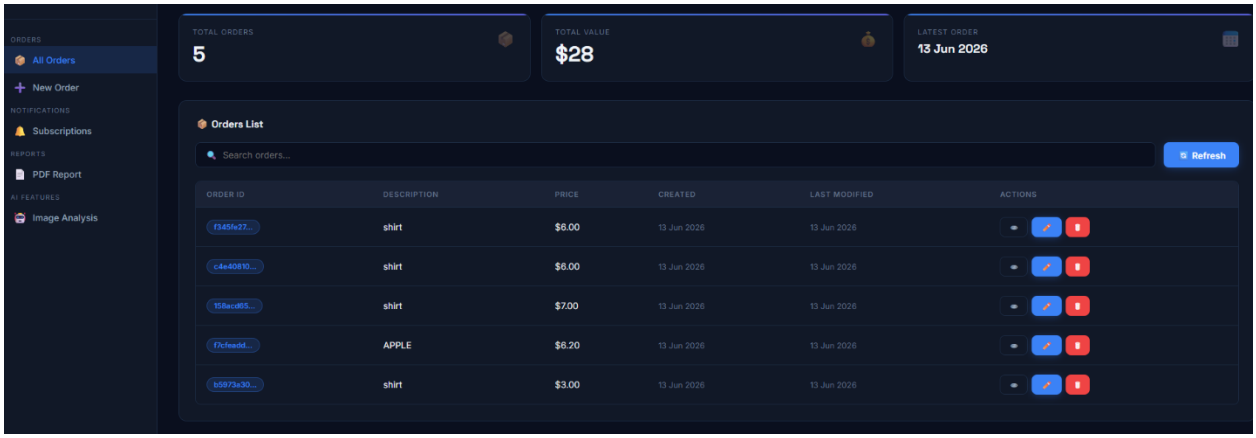




Flow 2: Get All Orders

Steps: Navigate to 'All Orders' tab → Click 'Refresh'

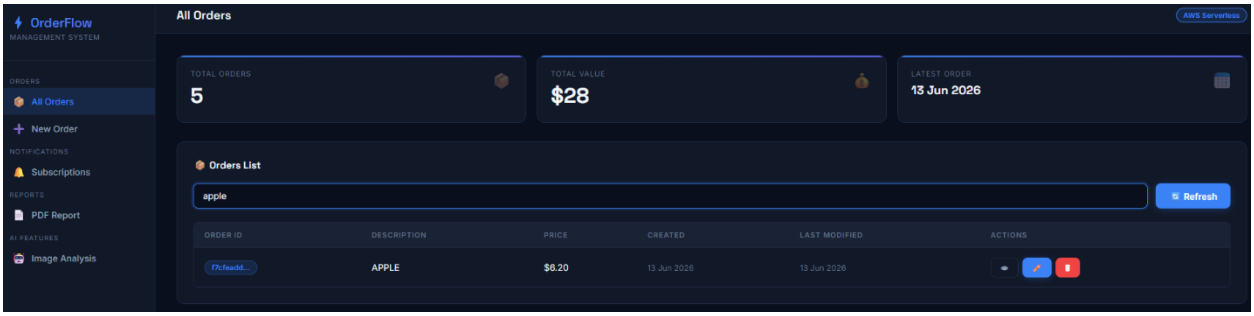
Expected Result: All orders are displayed in a table sorted by creation date.

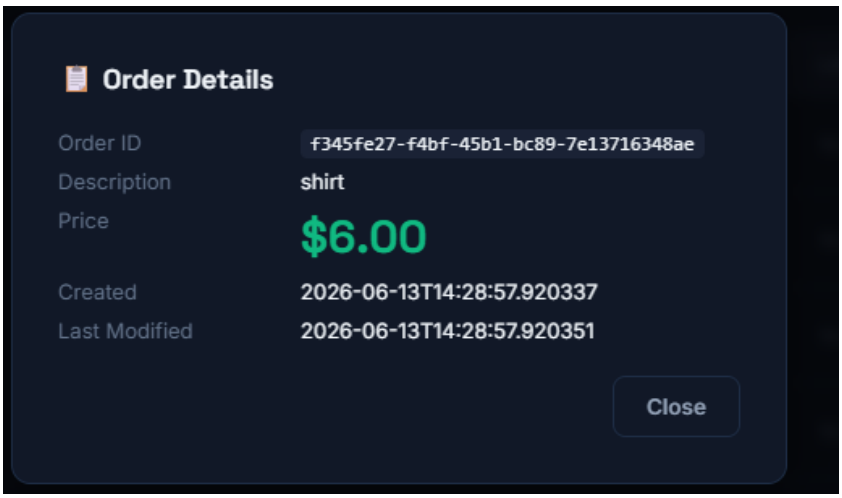


Flow 3: Get Specific Order

Steps: Click the eye icon ( ) on any order in the table. Alternatively, use the search bar to filter orders by description or order ID, then click the eye icon on the desired result.

Expected Result: A modal displays all details of the selected order.

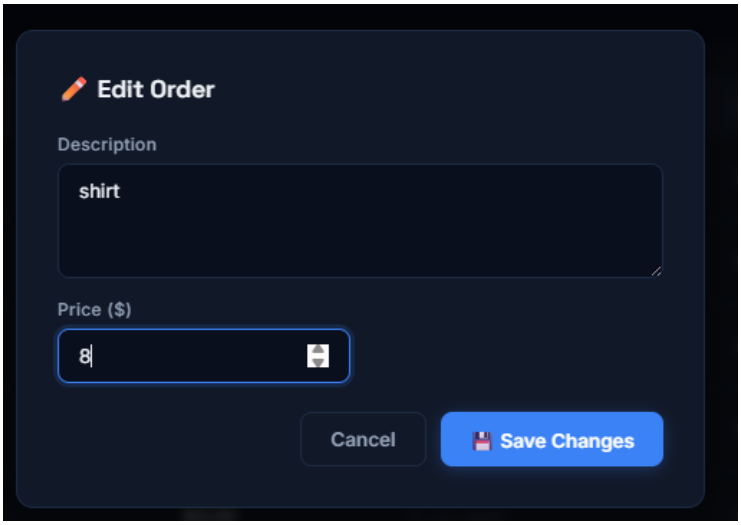




Flow 4: Update Order

Steps: Click the edit icon (🖍) on any order → Change price or description (one field is enough) → Click 'Save Changes'

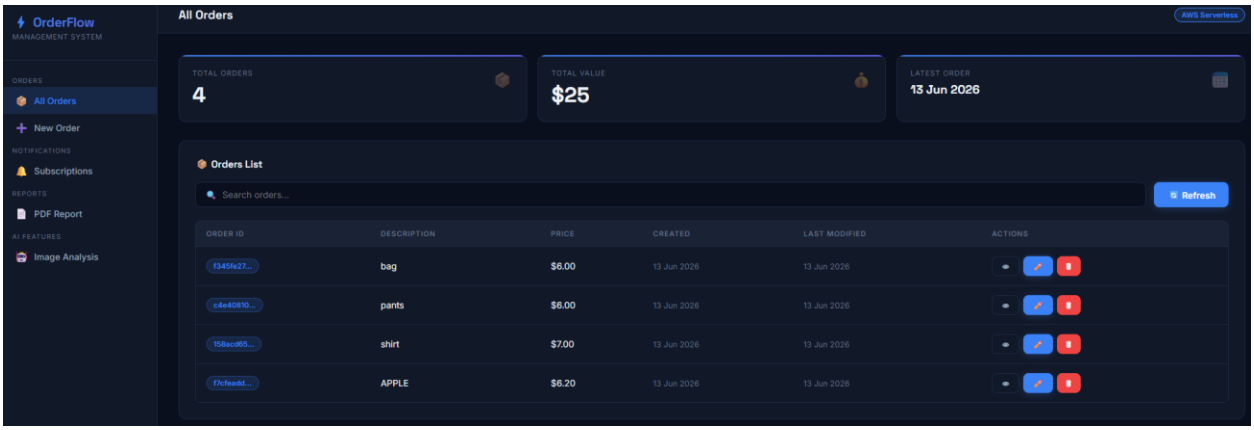
Expected Result: Order is updated. Only the changed field is modified. Last modified date is updated.

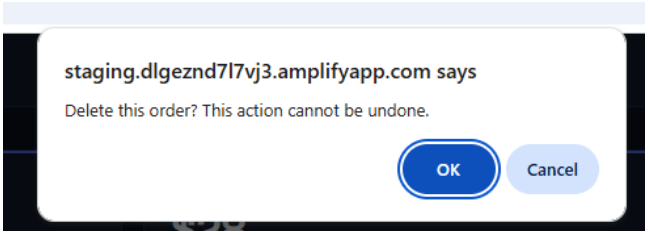


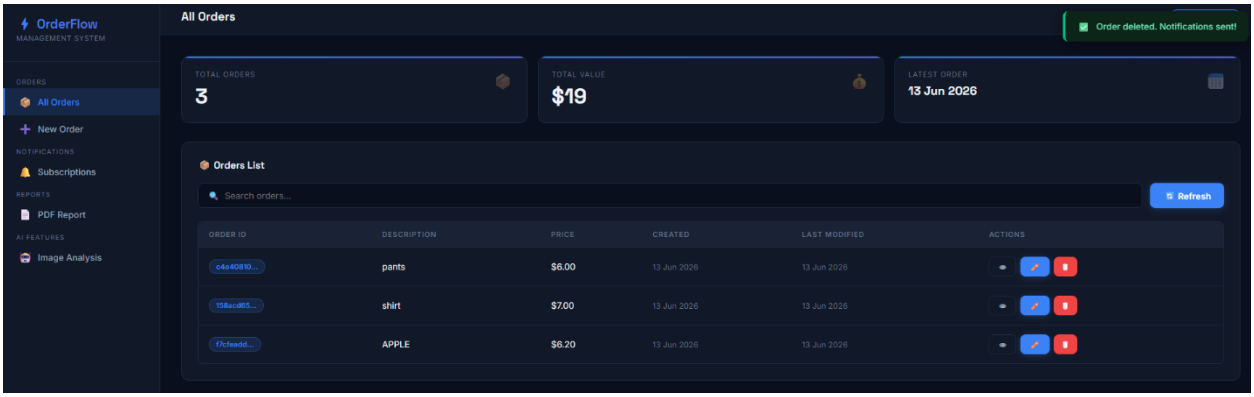
Flow 5: Delete Order with Notification

Steps: Subscribe with email → Confirm subscription from email → Delete an order → Check email inbox

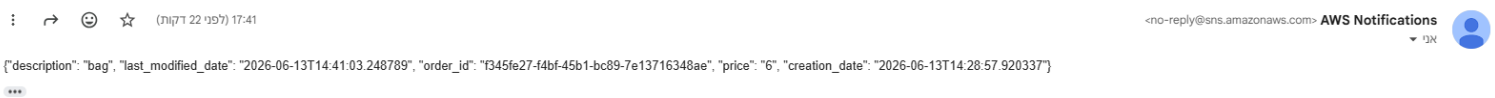
Expected Result: Order is deleted immediately. Email notification arrives with order details. Process is asynchronous via Step Functions.







From the mail



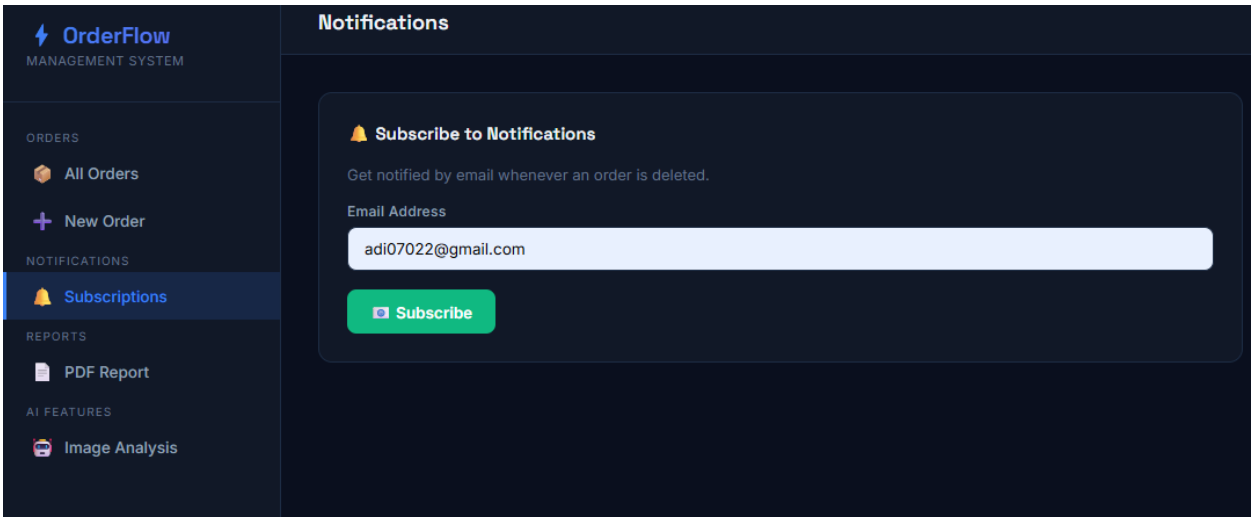
From the S3



Flow 6: Subscribe to Notifications

Steps: Navigate to 'Subscriptions' tab → Enter email → Click 'Subscribe'

Expected Result: Confirmation email is sent. After confirming, user receives notifications on order deletion.



You have chosen to subscribe to the topic:
arn:aws:sns:us-east-1:339713037456:order-notifications

To confirm this subscription, click or visit the link below (If this was in error no action is necessary):
Confirm subscription

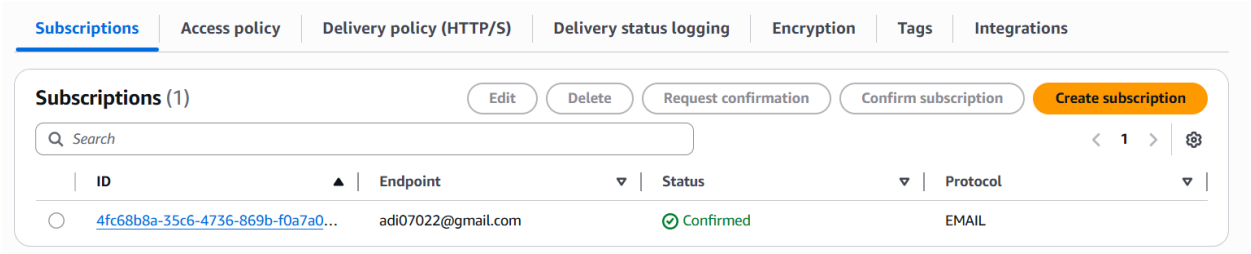
Please do not reply directly to this email. If you wish to remove yourself from receiving all future SNS subscription confirmation requests please send an email to sns-opt-out

Flow 7: Unsubscribe from Notifications

Steps: Navigate to 'Subscriptions' tab → Enter email → Click 'Unsubscribe'

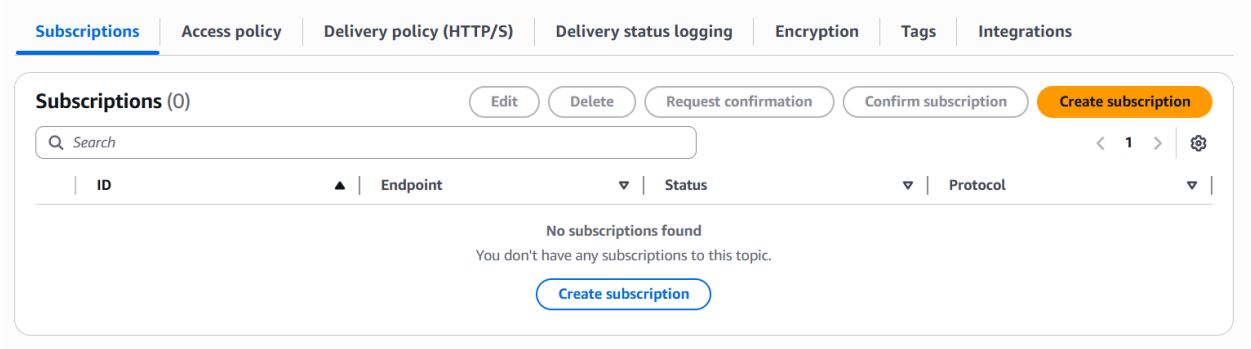
Expected Result: User is removed from the SNS topic and no longer receives notifications.

BEFORE-





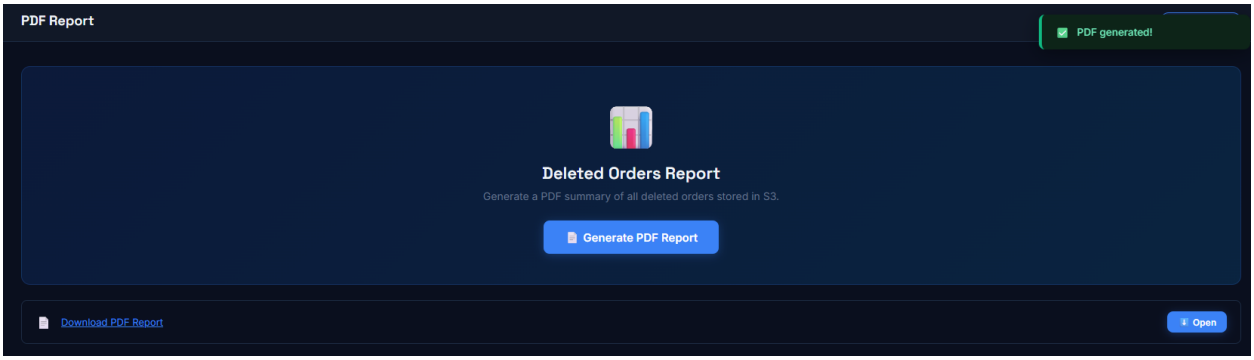
AFTER-

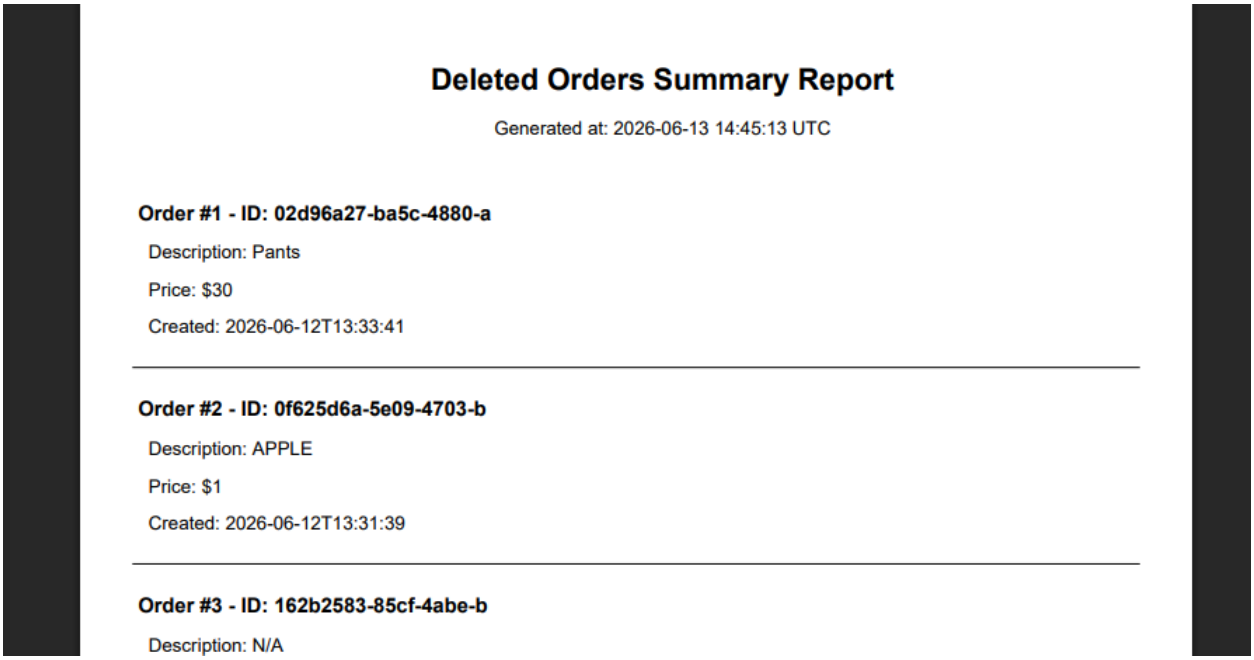


Flow 8: Generate PDF Report

Steps: Navigate to 'PDF Report' tab → Click 'Generate PDF Report'

Expected Result: A PDF is generated with all deleted orders, stored in S3, and a download URL is returned.





Deleted Orders Summary Report

Generated at: 2026-06-13 14:45:13 UTC

Order #1 - ID: 02d96a27-ba5c-4880-a

Description: Pants

Price: $30

Created: 2026-06-12T13:33:41

Order #2 - ID: 0f625d6a-5e09-4703-b

Description: APPLE

Price: $1

Created: 2026-06-12T13:31:39

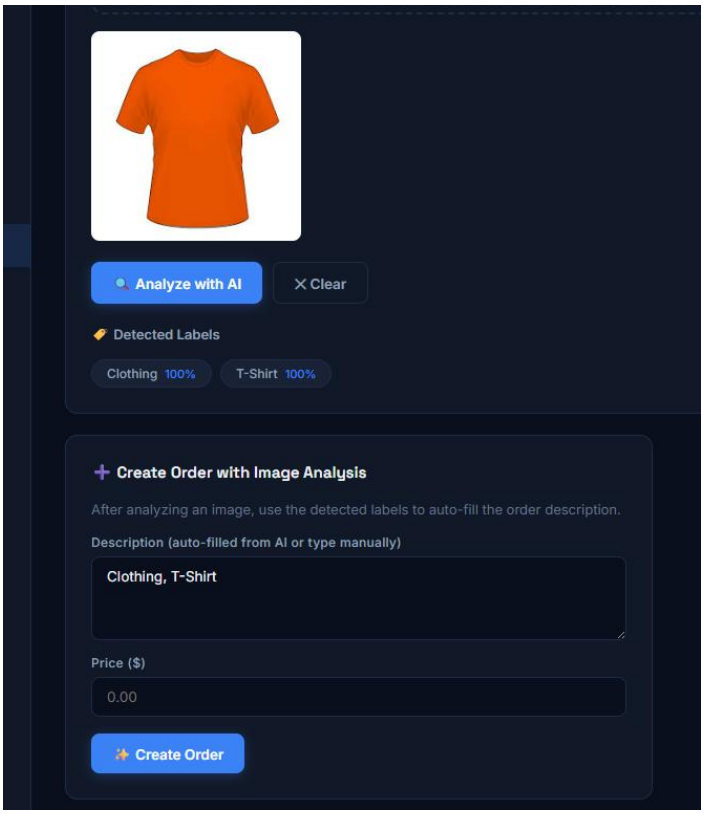
Order #3 - ID: 162b2583-85cf-4abe-b

Description: N/A

Flow 9: AI Image Analysis (Rekognition)

Steps: Navigate to 'Image Analysis' tab → Upload an image → Click 'Analyze with AI'

Expected Result: AWS Rekognition detects labels (e.g., Clothing 100%, Pants 95%) which auto-fill the order description.



5. Freestyle Enhancement — AWS Rekognition

Selected Service

AWS Rekognition — AI-powered image analysis service.

Why Rekognition?

We selected AWS Rekognition because it adds clear, demonstrable value to the Order Management System. Instead of manually typing order descriptions, users can upload a product image and let AI automatically detect what the item is. This makes order creation faster, more accurate, and showcases real-world AI integration within a business workflow.

What Functionality It Adds

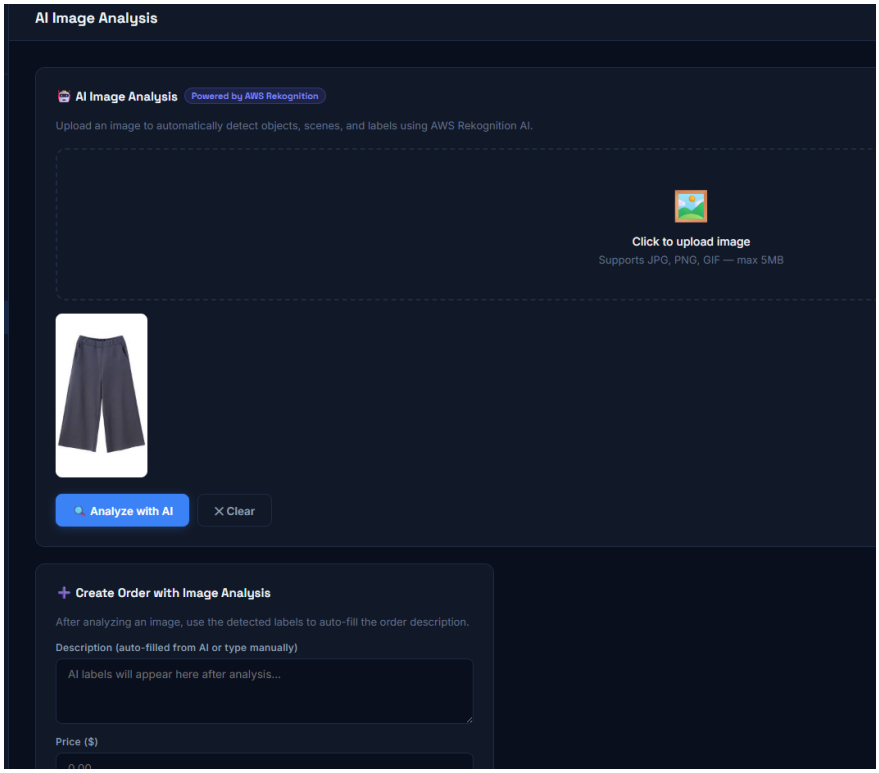
The Rekognition feature allows users to:

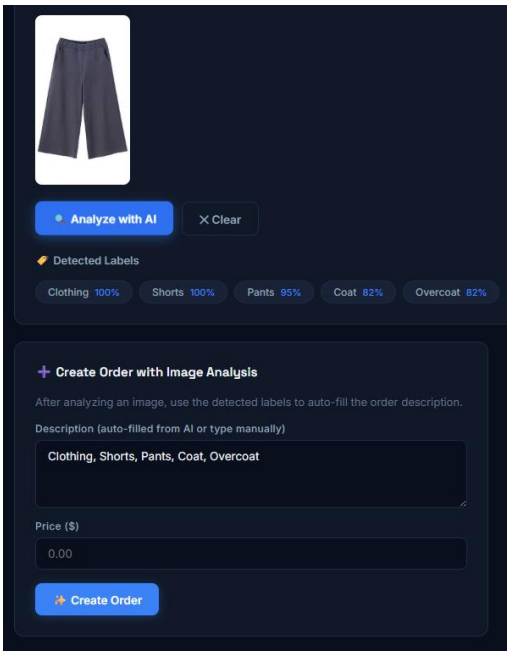
- Upload any product image (JPG, PNG, GIF)
- Automatically detect objects, scenes, and labels using AWS AI
- View detected labels with confidence percentages (e.g., Clothing 100%, Pants 95%)
- Auto-fill the order description with detected labels
- Create an order directly from the analyzed image

How to Execute and Verify

1. Navigate to the 'Image Analysis' section in the sidebar.
2. Click 'Click to upload image' and select any product image.
3. Click 'Analyze with AI' button.
4. Detected labels appear as tags with confidence percentages.
5. The description field is auto-filled with the detected labels.
6. Add a price and click 'Create Order' to create an order from the analysis.

Screenshot


AI Image Analysis
AI Image Analysis
Powered by AWS Rekognition
Upload an image to automatically detect objects, scenes, and labels using AWS Rekognition AI.
Click to upload image
Supports JPG, PNG, GIF — max 5MB
Analyze with AI
Clear
Create Order with Image Analysis
After analyzing an image, use the detected labels to auto-fill the order description.
Description (auto-filled from AI or type manually)
AI labels will appear here after analysis...
Price ($)


Analyze with AI
Clear
Detected Labels
Clothing 100%
Shorts 100%
Pants 95%
Coat 82%
Overcoat 82%
Create Order with Image Analysis
After analyzing an image, use the detected labels to auto-fill the order description.
Description (auto-filled from AI or type manually)
Clothing, Shorts, Pants, Coat, Overcoat
Price ($)
0.00
Create Order

6. Delete Order — Lambda Function Code

```
import json
import boto3
from boto3.dynamodb.conditions import Key

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('orders')
stepfunctions = boto3.client('stepfunctions')

STATE_MACHINE_ARN = 'arn:aws:states:us-east-1:339713037456:stateMachine:order-delete-process'

def lambda_handler(event, context):
    try:
        # Safely extract parameters
        path = event.get('pathParameters') or {}
        # Ensure this matches your API Gateway parameter (orderId or order_id)
        order_id = path.get('order_id')

        if not order_id:
            return response(400, 'Missing order_id parameter')

        # Using efficient Query (O(1)) instead of Scan (O(N))
        result = table.query(
            KeyConditionExpression=Key('order_id').eq(order_id)
        )
        items = result.get('Items', [])

        if not items:
            return response(404, 'Order not found')

        item = items[0]

        # Delete from DynamoDB
        table.delete_item(
            Key={
                'order_id': item['order_id'],
                'creation_date': item['creation_date']
            }
        )

        # Trigger Step Function asynchronously
        stepfunctions.start_execution(
            stateMachineArn=STATE_MACHINE_ARN,
```

```
            input=json.dumps({
                'message': json.dumps(item, default=str),
                's3Key': f"deleted-orders/{order_id}.txt"
            })
        )

        return response(200, 'Order deleted successfully and archiving process started.')

    except Exception as e:
        return response(500, f'Internal error: {str(e)}')

def response(status_code, body):
    return {
        'statusCode': status_code,
        'headers': {'Access-Control-Allow-Origin': '*'},
        'body': json.dumps(body, default=str)
    }
```